

Containers e Isolamento de Processos

Infraestruturas de Computação na Nuvem, Aula 04

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda (1/2)

- Containers vs máquinas virtuais.
- Primitivas Linux de isolamento: namespaces e cgroups.
- Union filesystems, imagens em camadas e as suas limitações.

Agenda (2/2)

- Arquitetura Docker: daemon, CLI, containerd, runc.
- Registries de imagens e versionamento por tags.
- Redes e volumes em Docker.
- Boas práticas e segurança de containers.
- Containers de aplicação vs containers de sistema.
- OCI e a convergência com máquinas virtuais leves.

Containers vs Máquinas Virtuais

- Containers partilham o kernel do sistema operativo do host, ao contrário das VMs, que virtualizam hardware e correm um kernel próprio.
- Overhead muito menor: sem hipervisor completo nem sistema operativo convidado duplicado.
- Arranque quase instantâneo, tipicamente em segundos (ou menos), em vez dos minutos típicos de uma VM.
- Contrapartida: isolamento menos forte do que numa VM, já que dependem inteiramente das primitivas de isolamento do kernel do host.

Namespaces Linux (1/2)

- Namespaces dão a cada processo uma visão isolada de um determinado recurso do sistema, criando a ilusão de máquina dedicada.
- **PID**: árvore de processos própria, isolada da do host.
- **NET**: pilha de rede própria (interfaces, rotas, portas).
- **MNT**: sistema de ficheiros e pontos de montagem próprios.

Namespaces Linux (2/2)

- **UTS**: hostname e domainname próprios.
- **IPC**: mecanismos de comunicação entre processos isolados (semáforos, filas de mensagens).
- **USER**: mapeamento de identificadores de utilizador e grupo, permitindo root dentro do container sem privilégios no host.
- Um container Docker é, na prática, um processo Linux normal ao qual foram atribuídos vários destes namespaces.

Cgroups: Limitação de Recursos

- Enquanto os namespaces isolam o que um processo vê, os cgroups (control groups) limitam o que um processo pode *consumir*.
- Limitação de CPU: quota e período de utilização atribuídos a um grupo de processos.
- Limitação de memória: teto máximo, com possibilidade de terminar o processo (OOM kill) se for excedido.
- Limitação de I/O: controlo de largura de banda de leitura/escrita em disco por grupo.

Cgroups: Versões e Funcionamento

- Os cgroups são expostos pelo kernel através de um sistema de ficheiros virtual, tipicamente em `/sys/fs/cgroup`.
- Escrever valores nesses ficheiros define os limites aplicados a um grupo de processos.
- **cgroups v1**: hierarquias separadas e independentes para cada tipo de recurso (CPU, memória, I/O).
- **cgroups v2**: hierarquia única e unificada, com um modelo de delegação mais coerente; é o modelo usado por omissão nas distribuições Linux recentes.
- Sem cgroups, um único container descontrolado poderia consumir toda a memória ou CPU do host.

Namespaces Isolam, Cgroups Limitam

namespaces
isolam o que o processo VÊ

PID — árvore de processos própria

NET — interfaces e portas próprias

MNT — sistema de ficheiros próprio

UTS — hostname próprio

IPC / USER

cgroups
limitam o que o processo CONSOME

CPU — quota de processamento

Memória — teto máximo (OOM kill)

I/O — largura de banda de disco

Expostos em `/sys/fs/cgroup`

Um container é um processo Linux normal com namespaces atribuídos e cgroups aplicados.

Isolamento Não é Segurança Absoluta

- Todos os containers de um host partilham o mesmo kernel: uma vulnerabilidade no kernel é uma superfície de ataque comum.
- **Capabilities**: o Linux divide os privilégios de root em capacidades individuais (ex. `CAP_NET_ADMIN`); um container deve receber apenas as estritamente necessárias.
- **Seccomp**: filtra as chamadas de sistema (*syscalls*) que um processo pode invocar, reduzindo a superfície exposta do kernel.
- **AppArmor** e **SELinux**: mecanismos de controlo de acesso obrigatório, que restringem o que cada processo pode aceder.
- Quando é exigido isolamento forte (ex. entre clientes diferentes), uma VM continua a ser mais segura do que um container.

Union Filesystems e Imagens em Camadas

- OverlayFS combina múltiplos diretórios (camadas) numa única vista de sistema de ficheiros.
- Uma imagem Docker é composta por várias camadas só de leitura, empilhadas umas sobre as outras.
- Ao correr um container, é adicionada uma camada de escrita (writable layer) no topo, específica dessa instância.
- Copy-on-write: alterações a ficheiros de camadas inferiores são copiadas para a camada de escrita antes de modificadas, preservando a imagem original.

OverlayFS na Prática: Limitações

- No host, cada container corresponde a um diretório em `/var/lib/docker/overlay2/<ID>/`: `merged` (a vista final, dentro do container) e `diff` (só as alterações dessa camada).
- OverlayFS não suporta todas as operações POSIX – é uma união de diretórios, não um sistema de ficheiros completo.
- Abrir um ficheiro para escrita depois de já estar aberto para leitura obriga a copiá-lo primeiro para a camada de escrita: os dois descritores deixam de apontar para o mesmo ficheiro.
- `rename` não é suportado diretamente: a aplicação tem de copiar e depois apagar o original, o que é mais lento e ocupa mais espaço.
- Consequência prática: aplicações com padrões de I/O intensivos beneficiam de

Arquitetura Docker (1/2)

- **Docker CLI**: interface de linha de comandos usada pelo utilizador.
- **dockerd**: daemon que recebe os pedidos da CLI (ou da API) e gere imagens, redes e volumes.
- **containerd**: runtime de alto nível, responsável pelo ciclo de vida dos containers (criação, execução, paragem).
- **runc**: runtime de baixo nível que efetivamente cria o container, aplicando namespaces e cgroups.

Arquitetura Docker (2/2)

- Um Dockerfile descreve, em passos declarativos, como construir uma imagem (imagem base, dependências, código, comando de arranque).
- Cada instrução do Dockerfile gera, em regra, uma nova camada da imagem.
- O comando `docker build` executa estes passos e produz a imagem final.
- A imagem resultante pode depois ser instanciada em um ou mais containers com `docker run`.

Registries de Imagens

- Um registry é um repositório de imagens Docker, de onde estas podem ser descarregadas (`pull`) ou para onde podem ser enviadas (`push`).
- Docker Hub é o registry público de referência, com imagens oficiais e da comunidade.
- Organizações usam também registries privados, para imagens internas ou proprietárias.
- Versionamento por tags: identificam versões específicas de uma imagem (ex.: `nginx:1.25`, `minha-app:latest`).

Redes em Docker

- **bridge**: rede virtual isolada no host, usada por omissão; os containers comunicam entre si e saem para o exterior por NAT.
- Numa rede bridge *personalizada*, o Docker fornece resolução de nomes: um container alcança outro pelo seu nome.
- **host**: o container usa diretamente a pilha de rede do host, sem isolamento de rede (sem namespace NET próprio).
- **none**: sem qualquer interface de rede, para cargas de trabalho totalmente isoladas.
- Publicação de portas (`-p 8080:80`) mapeia uma porta do host para uma porta do container.

Persistência: Volumes e Bind Mounts

- A camada de escrita de um container desaparece quando este é removido – por omissão, os dados não persistem.
- **Volumes**: geridos pelo Docker, guardados numa área própria do host; a forma recomendada de persistir dados.
- **Bind mounts**: mapeiam um diretório concreto do host para dentro do container; úteis em desenvolvimento, mas dependentes da estrutura do host.
- **tmpfs**: armazenamento apenas em memória, que nunca chega ao disco – adequado a dados sensíveis ou temporários.
- Separar dados do ciclo de vida do container é essencial para tratar containers como recursos descartáveis.

Boas Práticas na Construção de Imagens

- Usar imagens base minimalistas (ex. Alpine, ou variantes *slim*) para reduzir tamanho e superfície de ataque.
- **Multi-stage builds:** compilar numa fase intermédia e copiar apenas o resultado final para a imagem entregue.
- Ordenar as instruções do Dockerfile do menos ao mais volátil, para aproveitar a cache de camadas (ex. instalar dependências antes de copiar o código).
- `.dockerignore`: evita enviar ficheiros desnecessários para o contexto de build.
- Não incluir segredos (passwords, chaves) na imagem: ficam registados nas camadas e são recuperáveis.
- Correr a aplicação com um utilizador sem privilégios, em vez de root.

Containers de Aplicação vs de Sistema

- **Container de aplicação** (Docker): corre tipicamente um único processo/serviço; é efêmero e descartável.
- Não tem um sistema de init completo; o ciclo de vida do container é o ciclo de vida desse processo.
- **Container de sistema** (LXC, visto na Aula 03): comporta-se como uma máquina Linux completa, com `systemd` e múltiplos serviços.
- Ambos usam as mesmas primitivas do kernel (namespaces e cgroups) – a diferença está na forma de utilização, não na tecnologia de base.
- Nos laboratórios desta UC usamos os dois: LXC no Proxmox para máquinas de sistema, Docker para empacotar aplicações.

OCI: Open Container Initiative

- Iniciativa que define especificações abertas e neutras de fornecedor para formatos de containers.
- **image-spec**: define o formato de uma imagem de container, independente da ferramenta que a construiu.
- **runtime-spec**: define como um runtime deve criar e executar um container a partir dessa imagem.
- Garante portabilidade: a mesma imagem pode correr em diferentes runtimes compatíveis com OCI, como runc, crun ou gVisor.

Containers como Máquinas Virtuais Leves

- A fronteira entre container e VM (Aula 03) está a esbater-se: se o runtime e o formato de imagem seguem OCI, a implementação por baixo deixa de ser relevante para quem usa o container.
- **Kata Containers:** implementa a mesma interface OCI, mas cada "container" corre dentro de uma micro-VM leve (com o seu próprio kernel), combinando a portabilidade de Docker com o isolamento de hardware de uma VM.
- **gVisor:** outra abordagem híbrida – intercepta as *syscalls* do container num kernel de utilizador, sem chegar a expor o kernel real do host.
- Motivação: cargas multi-tenant (ex. funções de clientes diferentes no mesmo host) em que o isolamento fraco de um container tradicional é insuficiente.

Síntese da Aula (1/2)

- Containers partilham o kernel do host, oferecendo menor overhead e arranque quase instantâneo face a VMs.
- Namespaces isolam o que um processo vê (PID, NET, MNT, UTS, IPC, USER); cgroups limitam o que consome (CPU, memória, I/O).
- OverlayFS e imagens em camadas permitem reutilização eficiente de camadas comuns entre imagens, à custa de limitações POSIX (escrita concorrente, rename) que justificam o uso de volumes para I/O intensivo.

Síntese da Aula (2/2)

- Docker articula dockerd, containerd e runc para transformar um Dockerfile numa imagem e, depois, num container em execução.
- Registries (como o Docker Hub) distribuem imagens versionadas por tags; redes bridge e volumes tratam de comunicação e persistência.
- O isolamento de containers não é absoluto: capabilities, seccomp e AppArmor/SELinux reforçam-no, mas uma VM continua a isolar melhor – por isso soluções como Kata Containers e gVisor aproximam as duas abordagens.
- OCI garante portabilidade das imagens entre diferentes runtimes de containers.

Obrigada.

Questões e comentários